```
Article.includes(:comments).where(comments: { visible: true })
```

This would generate a query which contains a `LEFT OUTER JOIN` whereas the `joins` method would generate one using the `INNER JOIN` function instead.

```
SELECT "articles"."id" AS t0_r0, ... "comments"."updated_at" AS t1_r5
FROM "articles" LEFT OUTER JOIN "comments" ON "comments"."article_id" =
"articles"."id" WHERE (comments.visible = 1)
```

If there was no `where` condition, this would generate the normal set of two queries.

Using `where` like this will only work when you pass it a Hash. For SQL-fragments you need to use `references` to force joined tables:

```
Article.includes(:comments).where("comments.visible =
true").references(:comments)
```

If, in the case of this `includes` query, there were no comments for any articles, all the articles would still be loaded. By using `joins` (an INNER JOIN), the join conditions **must** match, otherwise no records will be returned.

14 Scopes

Scoping allows you to specify commonly-used queries which can be referenced as method calls on the association objects or models. With these scopes, you can use every method previously covered such as `where`, `joins` and `includes`. All scope methods will return an `ActiveRecord::Relation`object which will allow for further methods (such as other scopes) to be called on it.

To define a simple scope, we use the `scope` method inside the class, passing the query that we'd like to run when this scope is called:

```
class Article < ActiveRecord::Base
  scope :published, -> { where(published: true) }
end
```

This is exactly the same as defining a class method, and which you use is a matter of personal preference:

```
class Article < ActiveRecord::Base
  def self.published
    where(published: true)
  end
end
```

Scopes are also chainable within scopes:

```
class Article < ActiveRecord::Base
  scope :published,               -> { where(published: true) }
  scope :published_and_commented, -> { published.where("comments_count
> 0") }
end
```

To call this `published` scope we can call it on either the class:

```
Article.published # => [published articles]
```

Or on an association consisting of `Article` objects:

```
category = Category.first
category.articles.published # => [published articles belonging to this
category]
```

14.1 Passing in arguments

Your scope can take arguments:

```
class Article < ActiveRecord::Base
  scope :created_before, ->(time) { where("created_at < ?", time) }
end
```

Call the scope as if it were a class method: